FIT3080 Artificial Intelligence
Solution to Practice Questions in Agents and Problem Solving as Search

1. [**Agents**] Consider the vacuum agent presented in class, but assume that the room has 8 squares and a square can get dirty after it has been cleaned.

   (a) Use PEAS to specify the task environment.
   SOLUTION:
   Performance measure: cleanliness of the room, number of moves.
   Environment: room, furniture, people.
   Actuators: vaccum suction, wheels.
   Sensors: vision, touch.

   (b) Specify the attributes of the environment.
   SOLUTION:
   Known (the conditions of the environment are known)
   Stochastic (squares can become dirty at any time)
   Observable (everything can be seen)
   Multiple agents (there are people who may dirty the room)
   Sequential (next action depends on previous action)
   Static (the environment may change, but not while the robot is deciding)
   Continuous (the wheels of the robot may go in any direction, and stop at any point).

2. [**Agents**] Consider shopping for used AI books on the Internet.

   (a) Use PEAS to specify the task environment
   SOLUTION:
   Performance measure: time spent, money spent.
   Environment: network, electronic device.
   Actuators: Hands.
   Sensors: Eyes, Ears (if there are voices from the web page).

   (b) Specify the attributes of the environment.
   SOLUTION
   Partially observable (could not know all used AI books in the Internet)
   Deterministic (you click, the device responds)
   Sequential (next actions, e.g., open a webpage depends on previous action, e.g. click the link)
   Dynamic (the status of a book might change while the agent is thinking)
   Discrete (the actions have a finite number of distinct states: click the book, view the book, buy the book etc.)
   Single agent (there may be other users, but we don't interact with them)

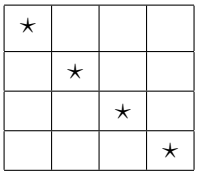
3. [**Backtrack**] Specify a state representation, operators and goal test to solve the N-queen problem with N=4:
Four queens have to be placed on a 4X4 board where no row will have more than one queen, no column will have more than one queen, and no diagonal will have more than one queen.
Illustrate the workings of the backtracking algorithm (Backtrack1) to solve this problem.

SOLUTION

Note: The solution is trying to align with what you've learned in the lecture, you can of course have a better representation and better operators to find a solution faster.

State representation:
An empty list of tiles [0000] to represent no queen is placing in the board. Each position in the list represents the row number (between 1-4) of the queen that is in column i.

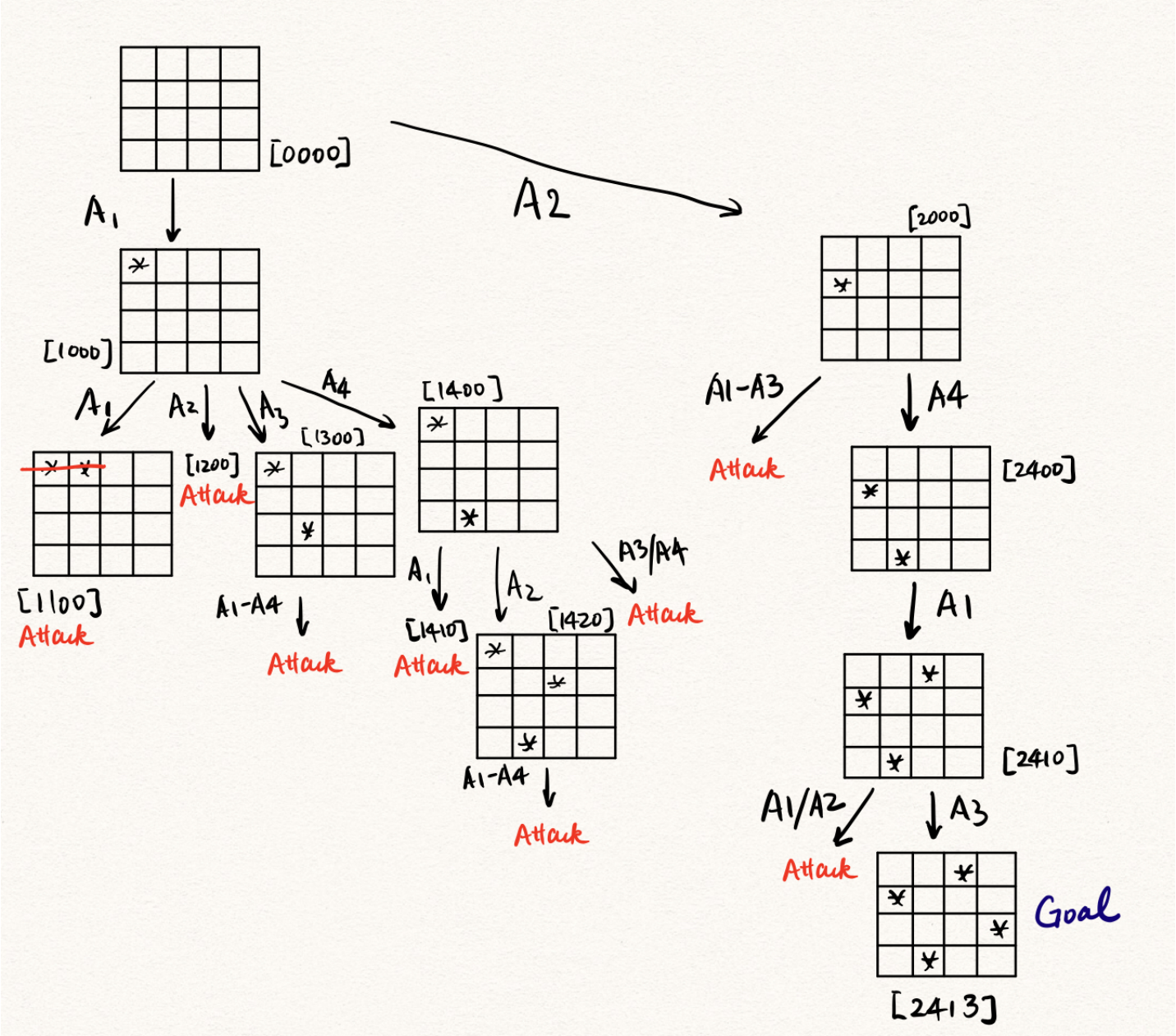
For example, a representation board = [1234] means a board with queens ($\star$) as below:



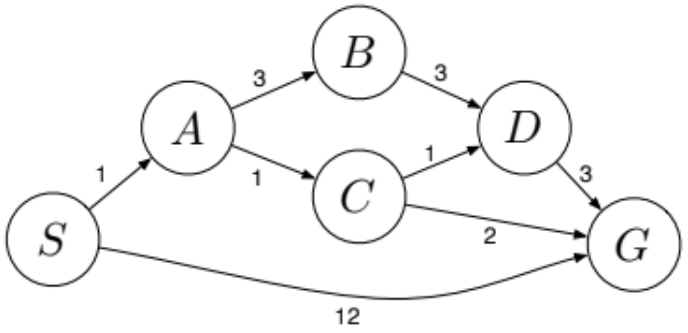
Operators:
Assign the tile value for the next unassigned queen, i.e., A_i means "assign the queen to row i, and to the first empty column". So A_i can be applied at most 4 times.

Goal test:
Check rows: All the numbers in the list are different.
Check columns: You don't need to do column check for this representation as no queen will be in the same column.
Check diagonals: Any pair of queen should have different row and column difference. If a queen is in row $r1$, column $c1$ and another queen in $r2$ and $c2$, then $|r1 - r2| \neq |c1 - c2|$ should hold for any pair of queens. For instance, if the board is [3124], the second queen ($r1 = 1$, $c1 = 2$) and the third queen ($r2 = 2$, $c2 = 3$) are attacking each other diagonally because $|1 - 2| = |2 - 3|$.

Backtrack illustration:


[0000]
A1
A2
[1000]
[2000]
A1
A2
A3
A4
A1-A3
A4
[1100]
Attack
[1200]
Attack
[1300]
[1400]
Attack
[2400]
A1-A4
Attack
A1
A2
A3/A4
[1410]
Attack
[1420]
Attack
A1
[2410]
A1-A4
Attack
A1/A2
A3
Attack
Goal
[2413]

4. [**GraphSearch**]



Answer the following questions about the search problem shown above, where S is the start and G is the goal. For the questions that ask for a path, please give your answers in the form '$S - ... - G$.'

(a) Breadth-first search
List the nodes according to their order of expansion. For each expansion, list FRONTIER (**with the nodes in the correct order**) and EXPLORED.
SOLUTION:
S FRONTIER={A, G}, EXPLORED={S}
G is generated, so no more expansions.

What path would BFS return for this search problem?
SOLUTION: S-G

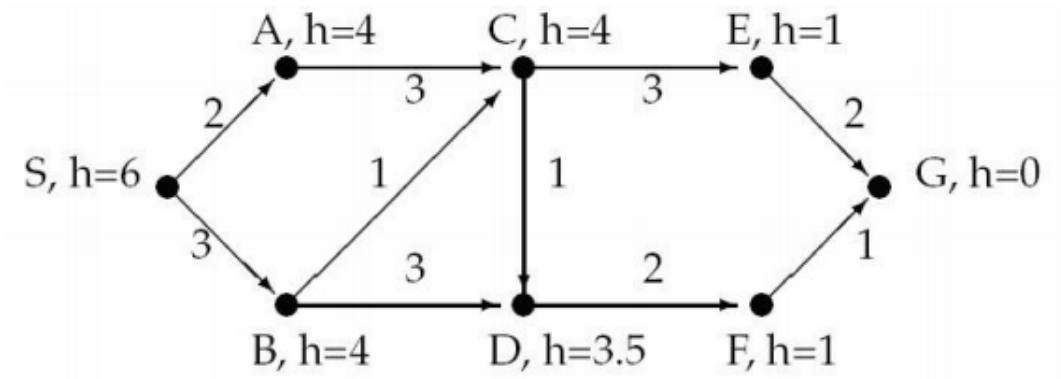
(b) Uniform cost search
List the nodes according to their order of expansion. For each expansion, list FRONTIER (**with the nodes in the correct order**) and EXPLORED.
SOLUTION:
S FRONTIER={A(1), G(12)}, EXPLORED={S(0)}
A FRONTIER={C(2), B(4), G(12)}, EXPLORED={S(0), A(1)}
C FRONTIER={D(3), G(min{12, **4**}), B(4) }, EXPLORED={S(0), A(1), C(2)}
NOTE: You can use strategies such as deeper path first when you are breaking ties.
D FRONTIER={G(min{12, **4**, 6}), B(4)}, EXPLORED={S(0), A(1), C(2), D(3)}
B FRONTIER={G(min{12, **4**, 6})}, EXPLORED={S(0), A(1), C(2), D(3), B(4)}
G FRONTIER={ }, EXPLORED={S(0), A(1), C(2), D(3), B(4), G(4)}

What path would uniform-cost return for this search problem?
SOLUTION: S-A-C-G

(c) Depth-first search
List the nodes according to their order of expansion. For each expansion, list FRONTIER (**with the nodes in the correct order**).
SOLUTION:
S FRONTIER={A(1), G(12)}
A FRONTIER={B(4), C(2), G(12)}
B FRONTIER={D(7), C(2), G(12)}
D FRONTIER={G(10), C(2)}
G FRONTIER={C(2)}

What path would DFS return for this search problem?
SOLUTION: S-A-B-D-G

5. [**GraphSearch**] Suppose we want to use algorithm A on the graph below to find the shortest path from node S to node G. Each node is labeled by a capital letter and the value of a heuristic function. Each edge is labeled by the cost to traverse that edge.



For this problem:

- Apply algorithm A to this graph, filling in the table below. Indicate the f, g and h values of each node in FRONTIER and EXPLORED as shown in the first two rows of the table.
  If you find a path to a node that is already in FRONTIER, update its cost (using the lower f value) instead of adding another copy of that node to FRONTIER.
  SOLUTION:

iteration	node expanded	FRONTIER	EXPLORED
0		S=0+6=6	
1	S	A=2+4=6; B=3+4=7	S
2	A	B=7, C=9	S, A
3	B	C=8, D=9.5	S, A, B
4	C	E=8, D=8.5	S, A, B, C
5	E	D=8.5, G=9	S, A, B, C, E
6	D	F=8, G=9	S, A, B, C, E, D
7	F	G=8	S, A, B, C, E, D, F
8	G		S, A, B, C, E, D, F, G

- Show the path found by algorithm A on the graph above.
  SOLUTION: S-B-C-D-F-G.

6. [**Algorithm A***] The evaluation function *f(n) = d(n) + W(n)*, where *d(n)* is the cost of arriving at node *n* and *W(n)* is the sum of Manhattan Distance for each misplaced tile, is used in conjunction with an A^* algorithm to search from the start node:

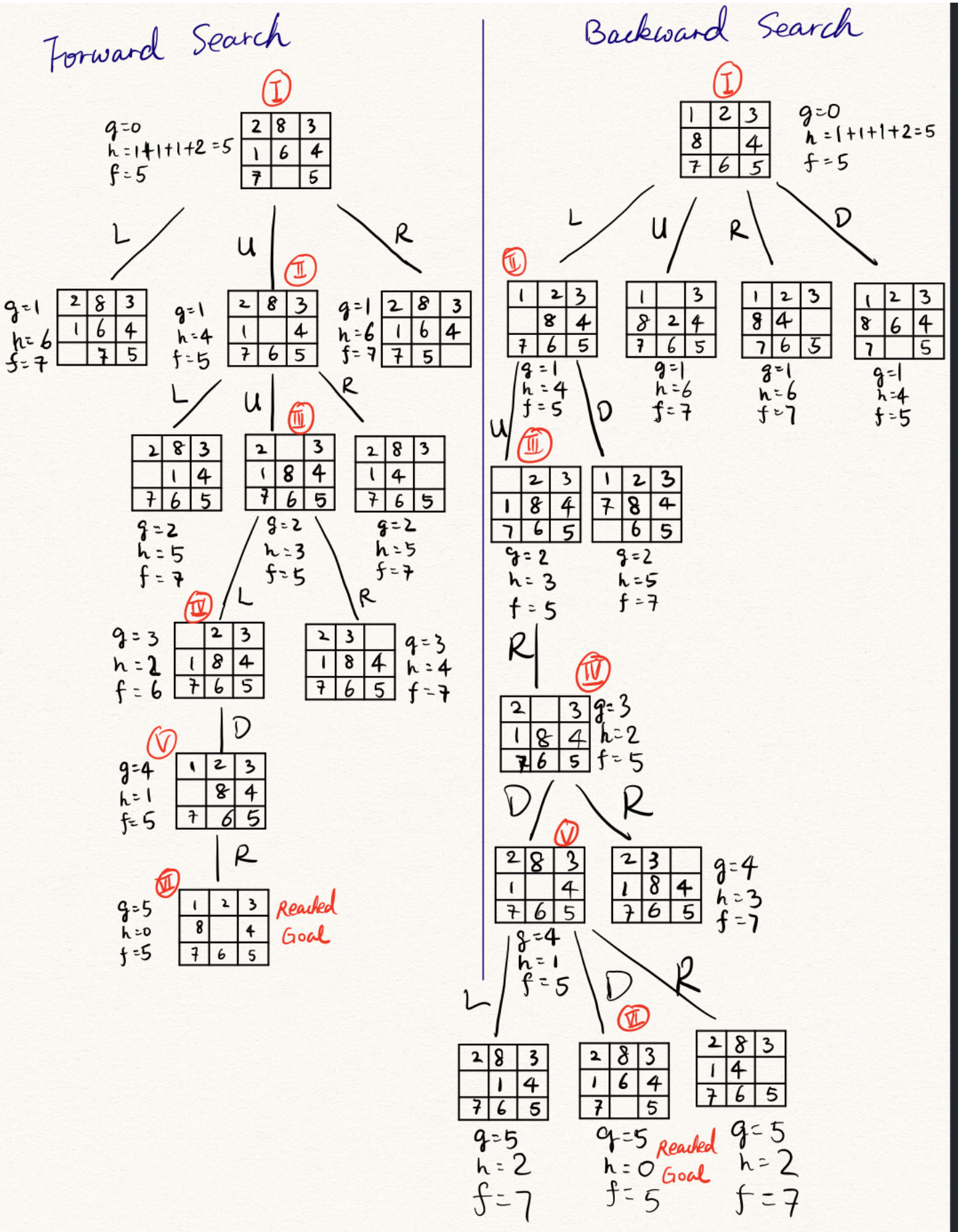
2	8	3
1	6	4
7		5

To the goal node:

1	2	3
8		4
7	6	5

Use this evaluation function to search forward (from start node to goal node) and backward (from the goal node to the start node). Where would the backward search meet the forward search?

SOLUTION:
From the search paths, we can see that Step III in Forward search met the same state of Step IV in Backward search. Similarly, Step IV in Forward search met the same state in Step III in Backward search.

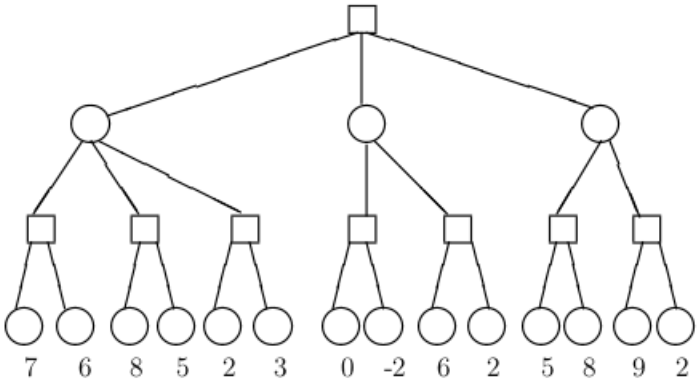

Forward Search
I
g=0
h=1+1+1+2=5
f=5
L
U
R
II
g=1
h=6
f=7
g=1
h=4
f=5
g=1
h=6
f=7
L
U
R
III
g=2
h=5
f=7
g=2
h=3
f=5
g=2
h=5
f=7
L
R
IV
g=3
h=2
f=6
g=3
h=4
f=7
D
V
g=4
h=1
f=5
R
VI
g=5
h=0
f=5
Reached Goal
Backward Search
I
g=0
h=1+1+1+2=5
f=5
L
U
R
D
II
g=1
h=4
f=5
g=1
h=6
f=7
g=1
h=6
f=7
g=1
h=4
f=5
U
D
III
g=2
h=3
f=5
g=2
h=5
f=7
R
IV
g=3
h=2
f=5
D
R
V
g=4
h=3
f=7
g=4
h=1
f=5
L
D
R
VI
g=5
h=2
f=7
g=5
h=0
f=5
Reached Goal
g=5
h=2
f=7

7. **[Search]** Which of the following are true and which are false? Explain your answers.

   (a) Depth-first search always expands at least as many nodes as A* search with an admissible heuristic.
   SOLUTION: False: a lucky DFS might expand exactly d nodes to reach the goal. A* dominates any graph-search algorithm that is guaranteed to find an optimal solution.

   (b) h(n) = 0 is an admissible heuristic for the 8-puzzle.
   SOLUTION: True: h(n) = 0 is always an admissible heuristic, since costs are nonnegative, and bounded from above by $h*$.

8. **[Game playing]** Consider the following game tree, where MAX plays in the square positions and MIN plays in the circles.

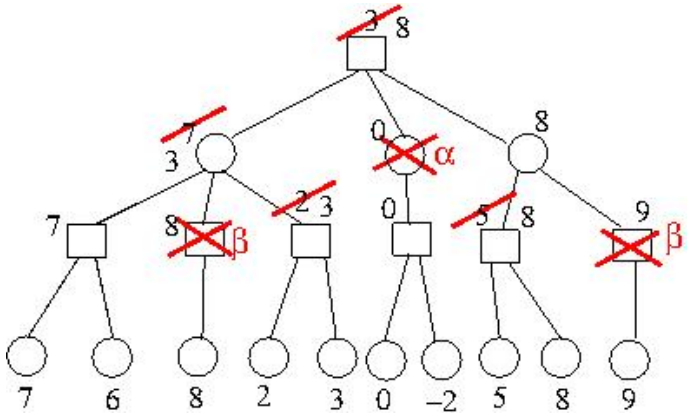


   (a) Conduct an α-β search of this game starting at the leftmost node to determine which move should MAX make. Draw the resulting game tree so that only the visited nodes appear in your diagram, i.e., **without** the nodes that are cut off. Indicate clearly the backed up value of each node, the updates performed on the backed up values, the α cut-offs and the β cut-offs you have performed.

   (b) What is the best move for MAX and what is its backed up value?

   SOLUTION:

   (a) α-β search. **Note: the solution is according to Nilsson, and may differ slightly from that produced by the algorithm in Russell and Norvig.**



   (b) Best move is to the right.
   Backed-up value is 8.

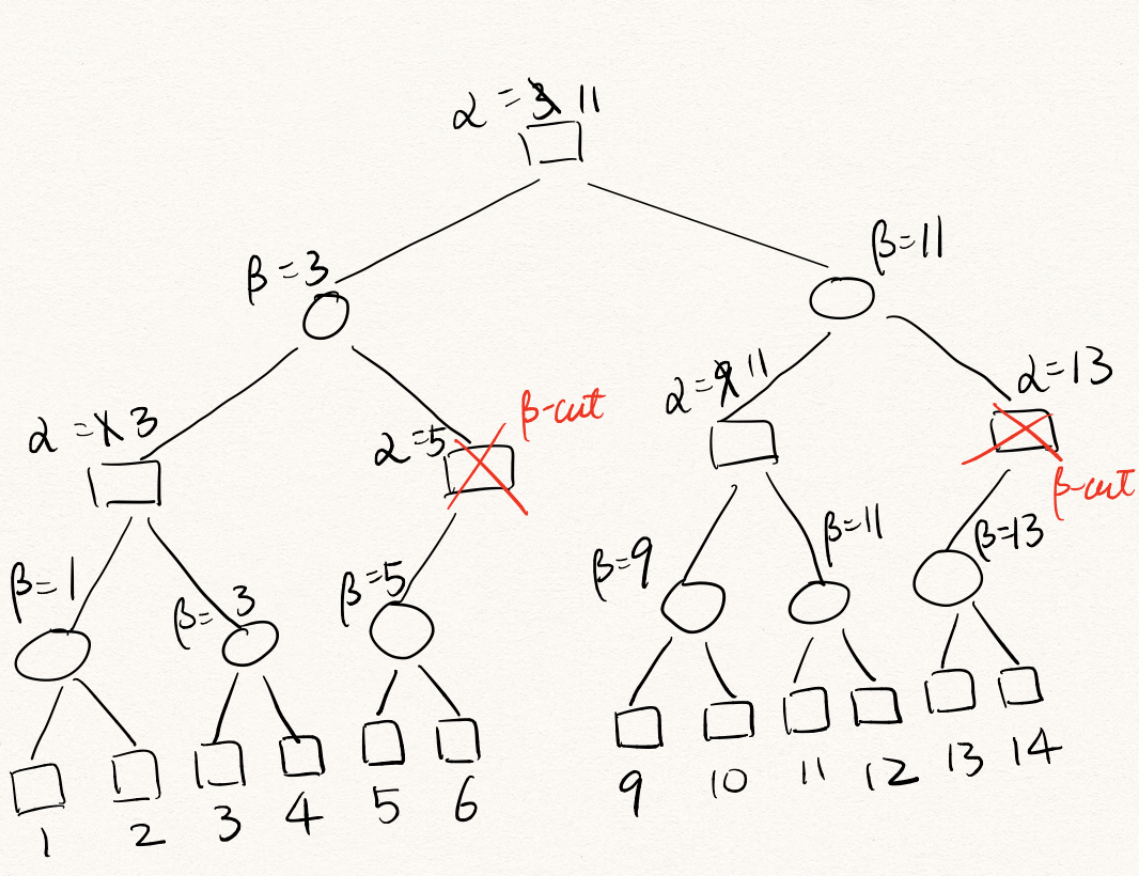
9. **[Game playing]** Consider a game tree with branching factor 2 and depth 5. Assume that it is MAX's turn to play, and that the evaluation function at the leaf nodes is in increasing order from left to right, such that its value for the leftmost node is 1, and for the rightmost node is 16 (the leaf nodes are MAX nodes).

   Conduct an α-β search of this game tree, starting from leftmost-node-first. In your α-β tree, clearly indicate the propagation of the α and β values, the performed cutoffs and the inspected leaf nodes.

   Upon completion of the search, state the final backed-up value of the root node and the recommended move (Left or Right). Also state the number of α and β cutoffs performed, and the number of leaf nodes generated.

   SOLUTION:

   - α-β search. **Note: the solution is according to Nilsson, and may differ slightly from that produced by the algorithm in Russell and Norvig.**



   - The final backed - up value is 11 and the recommended move is **to the right**. There were no α cut-offs, 2 β cut-offs, and 12 leaf nodes were generated.